Node.js Interview Notebook – Top 30 Questions

Oct 21, 2026 · @Hardik Sindhwani

1. Fundamentals (Q1–6)

Q1. What is Node.js? A JavaScript runtime built on Chrome's V8 engine that lets you run JS outside the browser. It uses an event-driven, non-blocking I/O model, which makes it good for APIs, real-time apps and I/O-heavy work.

Q2. Is Node single-threaded? How does it handle many requests? Your JavaScript runs on one main thread, but I/O is handed off to libuv (the C library behind Node), which uses the OS and a thread pool (default 4 threads). When the work finishes, the callback is queued and the event loop runs it. So one thread can serve thousands of connections as long as it is never blocked.

Q3. What are V8 and libuv?

- V8: compiles JavaScript to machine code.
- libuv: gives Node the event loop, the thread pool, and async file, network and DNS operations.

Q4. Blocking vs non-blocking? Blocking code stops everything until it finishes (`fs.readFileSync`). Non-blocking code starts the work and continues, and a callback, promise or event reports the result later (`fs.readFile`). Never use sync calls inside request handlers.

Q5. When is Node a bad choice? CPU-heavy work (image or video processing, heavy computation) blocks the event loop. Fix it with `worker_threads`, `child_process`, or a separate service.

Q6. `cluster` vs `worker_threads`?

- `cluster`: forks multiple Node processes, one per CPU core, sharing one port. It scales HTTP servers across cores.
- `worker_threads`: threads inside one process, sharing memory (`SharedArrayBuffer`). It is meant for CPU-heavy tasks.

2. Async, callback hell, event loop (Q7–15)

Q7. What is a callback? A function passed to another function, to be called when an operation finishes. Node uses the error-first convention: `callback(err, data)`.

```
fs.readFile('a.txt', 'utf8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

Q8. What is callback hell? (very likely question) Deeply nested callbacks, each depending on the previous result, so the code drifts right ("pyramid of doom"). It is hard to read, hard to handle errors in, and hard to maintain.

```
getUser(id, (err, user) => {
  getOrders(user.id, (err, orders) => {
    getItems(orders[0].id, (err, items) => {
      // ...and so on
    });
  });
});
```

Q9. How do you avoid callback hell?

1. Name your functions and keep them flat.
2. Use Promises with `.then()` chaining.
3. Use `async/await` (best option).
4. Use `Promise.all` for independent tasks.
5. Use `util.promisify` to convert old callback APIs.

```
async function load(id) {
  try {
    const user = await getUser(id);
    const orders = await getOrders(user.id);
    return await getItems(orders[0].id);
  } catch (err) {
    console.error(err);
  }
}
```

Q10. What are Promises? Their states? An object for a future value. States: *pending*, *fulfilled*, *rejected*. Once settled, a promise never changes state.

Q11. `Promise.all` vs `allSettled` vs `race` vs `any`?

- `all`: resolves when all succeed, rejects on the first failure.
- `allSettled`: waits for all and gives each result, never rejects.
- `race`: the first one to settle (success or failure) wins.
- `any`: the first one to succeed wins.

Q12. How does async/await work? It is syntax on top of promises. `await` pauses only that function, not the whole thread, and the rest of the code keeps running. Use `try/catch` for errors. Await independent calls together with `Promise.all`, not one by one.

Q13. Explain the event loop. The event loop is what lets Node do non-blocking work on one thread. It keeps cycling through phases and runs the callbacks queued in each:

1. **Timers**: `setTimeout`, `setInterval`
2. **Pending callbacks**: some system-level callbacks
3. **Poll**: I/O callbacks (files, network)
4. **Check**: `setImmediate`
5. **Close callbacks**: e.g. `socket.on('close')`

Between every phase, Node empties the `process.nextTick` queue first, then the promise microtask queue.

Q14. `process.nextTick` vs `setImmediate` vs `setTimeout(fn, 0)`?

- `nextTick`: runs right after the current operation, before the event loop continues (highest priority).
- Promise `.then`: microtask, runs right after `nextTick`.
- `setTimeout(0)`: timers phase.
- `setImmediate`: check phase, right after I/O.

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
console.log('sync');
// sync, nextTick, promise, timeout, immediate
```

The order of `timeout` and `immediate` in the main module can vary. Inside an I/O callback, `immediate` always comes first.

Q15. What happens if you block the event loop? No other request is served, timers are delayed, and the server looks frozen. Avoid sync APIs, big `JSON.parse` calls and heavy loops. Offload to worker threads, or split the work into chunks.

3. EventEmitter (Q16–19)

Q16. What is EventEmitter? A class in the `events` module that implements the observer pattern. Objects emit named events and listeners react to them. Much of Node's core (streams, http server, process) extends it.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('order', (id, qty) => {
  console.log(`Order ${id} x${qty}`);
});

emitter.emit('order', 101, 2); // Order 101 x2
```

Q17. Key methods?

- `on(event, fn)` / `addListener`: add a listener
- `once(event, fn)`: runs only the first time
- `emit(event, ...args)`: trigger the event, returns true if there were listeners
- `off(event, fn)` / `removeListener`: remove one listener
- `removeAllListeners(event)`: remove all
- `listenerCount(event)`: how many listeners

Q18. Is EventEmitter synchronous or asynchronous? Synchronous. `emit` calls all listeners in the order they were registered, immediately, before moving to the next line. To make a listener async, wrap its body in `setImmediate` or `process.nextTick`.

Q19. What is the special `error` event? What is the memory leak warning?

- If `emit('error')` has no listener, Node throws and the process can crash. Always add `emitter.on('error', ...)`.
- By default Node warns after 10 listeners on one event (`MaxListenersExceededWarning`). It usually means you keep adding listeners without removing them (a leak). Remove them with `off`, or raise the limit with `setMaxListeners(n)` only if it is intentional.

Bonus: a custom emitter class

```
class Logger extends EventEmitter {
  log(msg) {
    this.emit('message', msg);
  }
}
const logger = new Logger();
logger.on('message', m => console.log('LOG:', m));
logger.log('server started');
```

Likely follow-up: EventEmitter vs callbacks? A callback is called once, for one result. An emitter can fire many times and notify many listeners (one-to-many).

4. Modules, streams, buffers (Q20–24)

Q20. CommonJS vs ES Modules?

- CommonJS: `require()` / `module.exports`, loaded synchronously, the Node default.
- ESM: `import` / `export`, static and async loading, enabled with `"type": "module"` in package.json or `.mjs` files. ESM has no `__dirname`, and supports top-level `await`.

Q21. What is `package.json` and what is the difference between `dependencies` and `devDependencies`? package.json describes the project: name, scripts, dependencies. `dependencies` are needed in production. `devDependencies` are only for development (nodemon, jest, eslint). `package-lock.json` pins exact versions so installs are repeatable.

Q22. What are streams? Types? Streams process data in chunks instead of loading it all in memory, which is ideal for big files. Four types: **Readable**, **Writable**, **Duplex** (both), **Transform** (modifies data, e.g. gzip). Streams are EventEmitters too (`data`, `end`, `error`).

```
const fs = require('fs');
fs.createReadStream('big.txt')
  .pipe(fs.createWriteStream('copy.txt'));
```

Q23. What is a Buffer? A fixed-size chunk of raw binary memory outside the V8 heap, used for files, network packets and streams. Example: `Buffer.from('hi')`.

Q24. `fs.readFile` vs `createReadStream`? `readFile` loads the entire file into memory first, so it is fine for small files. `createReadStream` sends it in chunks, so memory stays low for large files.

5. Express, errors, security, performance (Q25–30)

Q25. What is middleware in Express? A function `(req, res, next)` that runs between the request and the response. It can modify `req` / `res`, end the request, or call `next()` to pass control on. Used for logging, auth, body parsing, and error handling.

```
app.use(express.json());
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});
```

Q26. How do you handle errors in Node?

- Callbacks: check `err` first.
- Promises: `.catch()`. async/await: `try/catch`.
- Express: an error middleware with four arguments `(err, req, res, next)`, placed last.
- Process level: listen for `unhandledRejection` and `uncaughtException` to log, then exit and let a process manager (PM2, Docker) restart.

Q27. `require` caching and `module.exports` vs `exports`? A module is loaded once and cached, so later `require` calls return the same object. `exports` is just a reference to `module.exports`. Assigning `exports = ...` breaks the link, so use `module.exports = ...` to export a single function or class.

Q28. How do you secure a Node/Express app? Use `helmet` for headers, rate limiting, validate and sanitize input, parameterised queries (avoid injection), `bcrypt` for passwords, JWT/session with expiry, CORS configured properly, secrets in environment variables (`.env`, never committed), and `npm audit` for dependencies.

Q29. How do you improve performance?

- Never block the event loop, use async APIs.
- Use `cluster` or PM2 to use all CPU cores.
- Cache with Redis.
- Use gzip compression, pagination, and database indexes.
- Use streams for big payloads.
- Put Nginx or a load balancer in front.

Q30. `process.env`, `__dirname`, and what is `npm` vs `npx`?

- `process.env`: access environment variables.
- `__dirname`: absolute path of the current file's folder (CommonJS).

- `npm` : installs and manages packages. `npx` : runs a package without installing it globally.

Last-minute checklist

In a 30–40 minute interview, expect callback hell, the event loop, EventEmitter, promises vs async/await, and middleware. Be able to write a short example of each from memory.

- [ ] Write callback hell and fix it with async/await (Q8, Q9)
- [ ] Explain the event loop phases and the output order of the Q14 snippet
- [ ] Write a small EventEmitter example with `on` , `once` , `emit` (Q16)
- [ ] Say that `emit` is synchronous (Q18)
- [ ] Explain why blocking the event loop is bad and how to avoid it (Q15)
- [ ] Know streams vs `readFile` (Q24)
- [ ] Know the 4 types of middleware use: logging, auth, parsing, errors (Q25)

Tips

1. Start with a one-line definition, then a tiny example, then a real use case.
2. If you don't know something, say what you do know and how you would find out.
3. Mention a project where you used these ideas. Interviewers like real experience.
4. Sleep. You have already prepared Java and React, so tonight just skim this once.